

SOCIOL 723: Social Statistics II

Week 12, Machine Learning: Regularization, Trees, and Neural Networks

Wenhao Jiang

Department of Sociology, Fall 2026



Today

The Bias–Variance Tradeoff

Regularization

Trees and Ensembles

Neural Networks

Beyond: Text, Images, and Foundation Models

Interpretation and Uncertainty

Prediction in Social Science

Practice

★ = essential, you must understand this

★ = for understanding, not examined

A Different Goal ★

Weeks 5–11 asked: *what is the causal effect, and what must be true to recover it?*
Today we set that aside entirely.

Prediction versus inference

Inference: we care about a parameter β , its interpretation, and its standard error.

Prediction: we care only about $\hat{Y}$ for units we have not seen. The parameters are means, not ends, and may not even be identified.

- These goals conflict. The model that predicts best is usually not the model whose coefficients you would want to interpret, and vice versa.
- Molina and Garip (2019): sociology has plenty of genuine *prediction* problems (who drops out, which neighbourhoods gentrify, which cases to audit), and separate ML tools for describing heterogeneity.
- Next week we put the two back together. Today, keep them apart.

Plan

Goal: predict Y for units we have not seen, and know how well we did.

1. The bias–variance decomposition, proved; why training error lies, by how much, and how cross-validation estimates what we want.
2. Ridge and lasso derived: closed forms, shrinkage through the SVD, soft thresholding, why a little bias always helps.
3. Trees, forests, boosting: the variance algebra behind averaging; boosting as gradient descent.
4. One benchmark on the course's GSS data, and what each method says about which variables matter.
5. Interpretation and uncertainty: importance, partial dependence, conformal prediction, and what changed with very flexible models.
6. Prediction in social science: the Fragile Families Challenge, bail decisions, a biased health algorithm, and fairness.

Running data: log earnings, GSS 2010–22, $n = 3,509$; script `figs/w12_gss.R`.

Bias-Variance

Decomposing Prediction Error ★

Let $Y = f(X) + \epsilon$ with $E[\epsilon | X] = 0$ and $\text{Var}(\epsilon | X) = \sigma^2$, and let $\hat{f}$ be estimated from a training sample drawn independently of the new observation. At a new point x_0 , taking expectations over *both* the training sample and the new outcome:

$$E[(Y_0 - \hat{f}(x_0))^2] = \underbrace{(E[\hat{f}(x_0)] - f(x_0))^2}_{\text{bias}^2} + \underbrace{\text{Var}(\hat{f}(x_0))}_{\text{variance}} + \underbrace{\sigma^2}_{\text{irreducible}}.$$

- **Bias:** how far the method is wrong on average, across training samples. Rigid models (a straight line) have high bias.
- **Variance:** how much $\hat{f}$ moves if you redraw the training data. Flexible models (a deep tree) have high variance.
- **Irreducible error:** noise. No method beats it.

Decomposing Prediction Error ★ (cont.)

- Every tuning parameter in this course is a dial between the first two terms. Penalty size, tree depth, number of neighbours, bandwidth (Week 9!), all the same dial.

The Decomposition, Proved

At a fixed x_0 , write $f = f(x_0)$ and $\hat{f} = \hat{f}(x_0)$; the new outcome is $Y_0 = f + \epsilon_0$ with ϵ_0 independent of the training data:

$$\begin{aligned}
 E[(Y_0 - \hat{f})^2] &= E[(\epsilon_0 + f - \hat{f})^2] && \text{substitute } Y_0 \\
 &= E[\epsilon_0^2] + 2 E[\epsilon_0] E[f - \hat{f}] + E[(f - \hat{f})^2] && \epsilon_0 \text{ independent of } \hat{f} \\
 &= \sigma^2 + E[(f - E\hat{f} + E\hat{f} - \hat{f})^2] && E[\epsilon_0] = 0; \text{ add and subtract } E\hat{f} \\
 &= \sigma^2 + (f - E\hat{f})^2 + E[(\hat{f} - E\hat{f})^2] && \text{cross term has mean zero}
 \end{aligned}$$

- Expectations are over *training samples*: bias and variance describe a method, not one fitted model.
- The same algebra gave the MSE of the RD estimator (Week 9) and the bandwidth choice; here the dial is model complexity.

Why Unbiasedness Stops Being the Goal ★

Weeks 1–2 prized unbiasedness: Gauss–Markov ranked estimators *within* the unbiased class.

For prediction, that is the wrong criterion

We care about total error. Accepting a little bias to remove a lot of variance lowers MSE. Every method today is deliberately biased.

- OLS with k regressors has variance growing in k : with p/n not small, the variance term swamps everything and OLS predicts badly even though it is unbiased.
- When $p > n$, OLS is not even defined, $\mathbf{X}'\mathbf{X}$ is singular.
- **In-sample fit is not evidence.** R^2 never falls when you add regressors, so training error is a systematically optimistic estimate of test error. The gap is the *optimism*, and it grows with model complexity.

How Optimistic Is Training Error?

OLS with p regressors (with intercept), fixed $\mathbf{X}$, $\text{Var}(\epsilon) = \sigma^2 \mathbf{I}$, hat matrix $\mathbf{H} = \mathbf{X}(\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'$, $\text{tr}(\mathbf{H}) = p$:

$$E\left[\frac{1}{n} \sum_i (Y_i - \hat{Y}_i)^2\right] = \frac{1}{n} E[\epsilon'(\mathbf{I} - \mathbf{H})\epsilon] = \sigma^2 \frac{n-p}{n} \quad \text{residual} = (\mathbf{I} - \mathbf{H})\epsilon; \text{ trace}$$

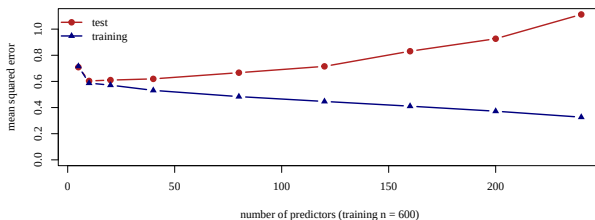
$$E\left[\frac{1}{n} \sum_i (Y_i^{\text{new}} - \hat{Y}_i)^2\right] = \sigma^2 + \frac{1}{n} E[\epsilon' \mathbf{H} \epsilon] = \sigma^2 \frac{n+p}{n} \quad \text{new noise at the same } \mathbf{X}$$

$$\text{optimism} = \frac{2p\sigma^2}{n} \quad \text{subtract}$$

- Training error is too low by $2p\sigma^2/n$: Mallows' C_p adds it back, and AIC (Week 3) is the likelihood version of the same correction.
- The penalty grows with p/n , not with p : 50 predictors are harmless with $n = 100,000$ and fatal with $n = 100$.

Optimism on the GSS

600 training cases; OLS on the first p columns of a 294-column design (main effects, pairwise interactions, squares); test on 1,053 held-out cases (figs/w12_gss.R).



p	10	20	40	80	120	200	240
test – training MSE	0.02	0.04	0.09	0.18	0.27	0.55	0.79
$2p\hat{\sigma}^2/n$	0.02	0.04	0.08	0.16	0.24	0.40	0.48

Up to $p \approx 80$ the formula is close; beyond it the test cases' new $\mathbf{X}$ add variance the fixed- $\mathbf{X}$ formula ignores.

Estimating Test Error: Cross-Validation ★

Split the data, fit on part, evaluate on the rest.

K -fold cross-validation

Partition the sample into K folds. For $k = 1, \dots, K$: fit on all folds but k , predict fold k , record the error. Average:

$$CV_K = \frac{1}{K} \sum_{k=1}^K \frac{1}{|\mathcal{I}_k|} \sum_{i \in \mathcal{I}_k} \left(Y_i - \hat{f}^{(-k)}(X_i) \right)^2.$$

- $K = 5$ or 10 is standard: leave-one-out ($K = n$) has low bias but high variance and is expensive.

Estimating Test Error: Cross-Validation ★ (cont.)

- Everything data-dependent must happen inside the fold, variable selection, standardization, imputation. Selecting predictors on the full sample and then cross-validating is a very common and very serious leak.
- With dependence (panels, clusters, time) split on the independent unit, not on rows. Same rule as the bootstrap in Week 2.

What Cross-Validation Estimates (Bates, Hastie, and Tibshirani 2024)

- CV refits the model K times on different training sets, so it estimates the **average** test error of the *procedure* over training sets of size $n(K - 1)/K$, not the error of the one model you finally fit.
- The usual interval $\widehat{CV} \pm 2 \widehat{se}$, treating the n held-out errors as independent, undercovers: the folds share training data, so the errors are correlated. **Nested CV** fixes the interval.
- **Tuning and evaluating are two jobs.** If λ is chosen by CV, the CV error at the chosen λ is optimistic; report error on a held-out test set never used for tuning, or nest the tuning inside each fold.
- **One-standard-error rule:** among λ within one se of the minimum, take the simplest. GSS: lasso keeps 48 of 294 terms at the minimum, 12 under the 1-se rule; test R^2 0.344 vs. 0.333.

Regularization

Ridge Regression ★

Add a penalty on the squared size of the coefficients:

$$\hat{\beta}^{\text{ridge}} = \arg \min_{\beta} \sum_{i=1}^n (Y_i - X_i' \beta)^2 + \lambda \sum_{j=1}^p \beta_j^2, \quad \hat{\beta}^{\text{ridge}} = (\mathbf{X}'\mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}'\mathbf{y},$$

where, as always with a penalty, $\mathbf{X}$ and $\mathbf{y}$ are centred and the intercept is left out of both the penalty and the displayed matrix.

- Adding $\lambda \mathbf{I}$ makes the matrix invertible **even when $p > n$** , which is why ridge works where OLS does not exist.
- Coefficients shrink toward zero but never reach it: ridge does not select.
- **Always standardize the predictors first**; the penalty is not scale-invariant, and the intercept is not penalized.

Ridge Regression ★ (cont.)

- Ridge shines under **dense** signals: many small effects, and correlated predictors whose coefficients OLS cannot separate.
- As $\lambda \rightarrow 0$ we recover OLS; as $\lambda \rightarrow \infty$ all slopes go to zero and the fit collapses to the sample mean of Y . The dial is the bias–variance dial.

Ridge Through the Singular Value Decomposition

Write $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}'$ with singular values $d_1 \geq \dots \geq d_p$ and columns $\mathbf{u}_j$ of $\mathbf{U}$:

$$\mathbf{X}\hat{\beta}^{\text{OLS}} = \mathbf{U}\mathbf{U}'\mathbf{y} = \sum_j \mathbf{u}_j (\mathbf{u}_j'\mathbf{y}) \quad \text{projection on the column space}$$

$$\mathbf{X}\hat{\beta}^{\text{ridge}} = \mathbf{X}(\mathbf{X}'\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}'\mathbf{y} = \sum_j \mathbf{u}_j \frac{d_j^2}{d_j^2 + \lambda} (\mathbf{u}_j'\mathbf{y}) \quad \mathbf{X}'\mathbf{X} = \mathbf{V}\mathbf{D}^2\mathbf{V}'$$

- Ridge shrinks each principal direction by $d_j^2/(d_j^2 + \lambda)$: directions in which the predictors barely vary (d_j small, the collinear ones) are shrunk most. That is where OLS variance comes from.
- **Effective degrees of freedom:**
 $\text{df}(\lambda) = \text{tr} [\mathbf{X}(\mathbf{X}'\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}'] = \sum_j d_j^2/(d_j^2 + \lambda)$, from p at $\lambda = 0$ down to 0.
 Plug it into the optimism formula.

A Little Bias Always Helps (Hoerl and Kennard 1970)

One centred regressor, $s = \sum_i x_i^2$; ridge gives $\hat{\beta}_\lambda = \hat{\beta} s / (s + \lambda)$:

$$\text{bias}(\hat{\beta}_\lambda) = -\frac{\lambda}{s + \lambda} \beta, \quad \text{Var}(\hat{\beta}_\lambda) = \left(\frac{s}{s + \lambda}\right)^2 \frac{\sigma^2}{s} \quad \text{shrink a constant fraction}$$

$$\text{MSE}(\lambda) = \frac{\lambda^2 \beta^2 + s \sigma^2}{(s + \lambda)^2} \quad \text{bias}^2 + \text{variance}$$

$$\left. \frac{d \text{MSE}}{d\lambda} \right|_{\lambda=0} = -\frac{2\sigma^2}{s^2} < 0 \quad \text{differentiate, evaluate at 0}$$

- Moving away from OLS *always* lowers MSE at first, whatever β is: the variance falls at first order, the bias enters squared. The best $\lambda^* = \sigma^2 / \beta^2$ (set the derivative to zero).
- The same is true with many regressors: some $\lambda > 0$ beats OLS. Unbiasedness is a choice, not an optimum.

Lasso ★

Penalize the *absolute* size instead:

$$\hat{\beta}^{\text{lasso}} = \arg \min_{\beta} \sum_{i=1}^n (Y_i - X_i' \beta)^2 + \lambda \sum_{j=1}^p |\beta_j|.$$

- The ℓ_1 penalty sets coefficients exactly to zero, so lasso performs variable selection and estimation simultaneously.
- Geometrically: the ℓ_1 constraint region is a diamond with corners on the axes, and the contours of the sum of squares typically first touch it at a corner. The ℓ_2 region is a ball with no corners, hence no exact zeros.
- Lasso suits **approximate sparsity**: a few predictors matter, the rest are approximately irrelevant.
- **Elastic net** mixes them, $\lambda(\alpha \sum |\beta_j| + \frac{1-\alpha}{2} \sum \beta_j^2)$, which handles groups of correlated predictors better than pure lasso.

Three Penalties, One Coefficient

Orthonormal design ($\mathbf{X}'\mathbf{X} = \mathbf{I}$), OLS estimate b_j . Minimize $(b_j - \beta_j)^2 + \text{penalty}$ coefficient by coefficient:

$$\text{ridge, } \lambda\beta^2 : \hat{\beta}_j = \frac{b_j}{1 + \lambda}$$

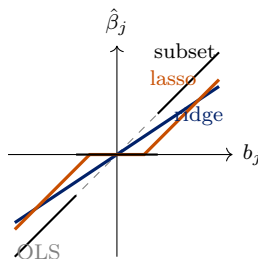
$$\text{lasso, } \lambda|\beta| : \hat{\beta}_j = \text{sign}(b_j)(|b_j| - \frac{\lambda}{2})_+$$

$$\text{best subset : } \hat{\beta}_j = b_j \mathbb{1}\{|b_j| > \sqrt{\lambda}\}$$

Lasso: for $\beta_j > 0$ the derivative

$-2(b_j - \beta_j) + \lambda = 0$ gives $b_j - \lambda/2$; if that is

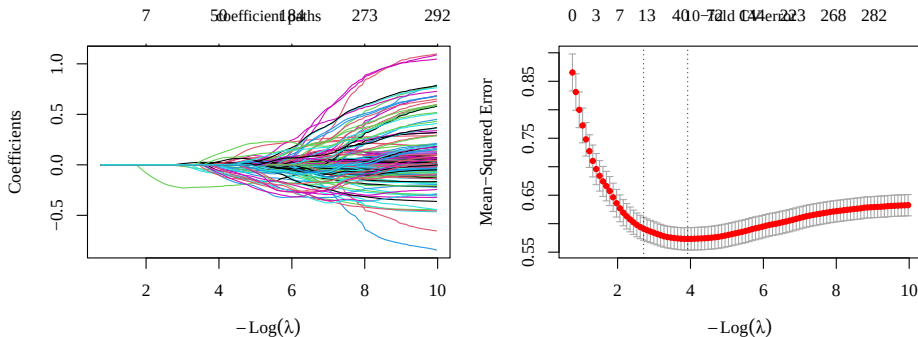
negative the minimum sits at 0.



- Ridge scales everything down; lasso subtracts a constant and stops at zero (**soft thresholding**); best subset keeps or kills (hard).
- Lasso's degrees of freedom are, on average, the number of nonzero coefficients (Zou, Hastie, and Tibshirani 2007).

The Lasso Path on the GSS

294 terms (main effects, interactions, squares), 2,456 training cases (figs/w12_gss.R):



- Left: coefficients enter one by one as λ falls. Right: CV error; the dashed lines mark the minimum (48 terms) and the 1-se choice (12 terms).

Choosing λ , and a Warning About Interpretation ★

- **Cross-validation** over a grid of λ : pick the minimizer, or the “one standard error” rule, the largest λ within one standard error of the minimum, which buys a simpler model at almost no predictive cost.
- There are also **theoretically derived** penalties that set λ from the model rather than by cross-validation. They matter when the lasso is a *step inside* an inference procedure rather than the final answer.

Lasso selection is not a variable-importance ranking

Among correlated predictors, lasso picks one essentially arbitrarily and zeroes the rest. Re-run on a bootstrap sample and you may get a different set. **Do not interpret the selected set as “the variables that matter”**, and do not report post-lasso p -values from a naive refit, the selection invalidates them. Valid post-selection inference is exactly what Week 13 is for.

Selection Is Not Importance, on the GSS

Lasso on the 25 main-effect columns, 100 bootstrap samples, 1-se rule (figs/w12_gss.R):

variable	share of samples selected	variable	share
education, age, female, SEI	1.00	parents' education	0.78
hours worked	1.00	vocabulary score	0.58
ever married	0.99	Black	0.26
year 2022	0.80	occupational prestige	0.01

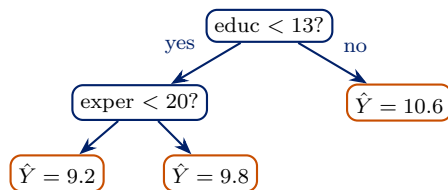
- Prestige and SEI correlate at 0.82. Lasso keeps SEI every time and prestige once in 100: not because prestige does not predict earnings, but because SEI already carries that information.
- A random forest's permutation importance ranks prestige third (after SEI and education). Two tools, two stories, same data.
- Middle-frequency variables (vocabulary, parents' education) flip in and out: the selected set has sampling variability (Mullainathan and Spiess 2017).

Trees

Regression Trees ★

Ridge and lasso control complexity, but both still start from an *additive linear* rule: you choose the variables, and an interaction exists only if you type it. Trees are the first method we meet that finds nonlinearities and interactions on its own.

Recursively split the covariate space to minimize within-node sum of squares; the prediction in a leaf is the mean of the training outcomes there.



- **Strengths:** automatic interactions and nonlinearity; invariant to monotone transformations; handles mixed variable types; readable.

Regression Trees ★ (cont.)

- **Weaknesses:** high variance, a small change in the data can change the first split and hence everything below it. A single deep tree overfits badly; a shallow one underfits.
- Pruning with a complexity penalty helps but does not solve it. The real fix is to average many trees.

Growing and Pruning a Tree

Grow: at each node, over every variable j and cut point s , split into $R_1 = \{x_j < s\}$ and $R_2 = \{x_j \geq s\}$ to minimize

$$\sum_{i \in R_1} (Y_i - \bar{Y}_{R_1})^2 + \sum_{i \in R_2} (Y_i - \bar{Y}_{R_2})^2;$$

repeat in each child until nodes are small. Greedy: each split is best given the splits above it, not best overall.

Prune: among subtrees T of the big tree, minimize

$$C_\alpha(T) = \sum_{\text{leaves } m} \sum_{i \in R_m} (Y_i - \bar{Y}_{R_m})^2 + \alpha |T|,$$

with $|T|$ the number of leaves and α by cross-validation: the lasso idea, penalizing size.

GSS: a CV-pruned tree reaches test R^2 0.313, below OLS (0.338). One tree is too rough a step function for a mostly smooth relationship.

Bagging, Random Forests, Boosting ★

- **Bagging:** fit trees on bootstrap resamples and average. Averaging B nearly independent estimates cuts variance by roughly $1/B$ without adding bias. But bootstrap trees are *correlated*, they all split on the same dominant predictor first, which caps the gain.
- **Random forests:** bag, but at each split consider only a random subset of m predictors. Decorrelating the trees is what makes the averaging work. m is the main tuning parameter, alongside node size. Out-of-bag error is a free cross-validation estimate.
- **Boosting:** fit trees *sequentially*, each on the residuals of the current ensemble, adding a shrunk version: $\hat{f} \leftarrow \hat{f} + \eta \hat{t}$. Attacks *bias* rather than variance, and with small η and many trees is usually the strongest tabular predictor available.

Bagging and random forests reduce variance; boosting reduces bias. That is the cleanest way to keep them straight.

Why Decorrelating Trees Works

B trees, each with variance σ^2 at a point, pairwise correlation ρ :

$$\begin{aligned}\text{Var}\left(\frac{1}{B}\sum_{b=1}^B T_b\right) &= \frac{1}{B^2}\left(\sum_b \text{Var}(T_b) + \sum_{b \neq b'} \text{Cov}(T_b, T_{b'})\right) && \text{variance of a sum} \\ &= \frac{1}{B^2}(B\sigma^2 + B(B-1)\rho\sigma^2) && B(B-1) \text{ ordered pairs} \\ &= \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 && \text{rearrange}\end{aligned}$$

- More trees kill the second term only. The first, $\rho\sigma^2$, is a floor: bagged trees on the same data are correlated.
- Random forests lower ρ by letting each split see only m of the p variables; each tree gets worse, the average gets better.
- Out-of-bag error: each tree skips about a third of the data $((1 - 1/n)^n \approx e^{-1})$.
GSS: OOB MSE 0.547, test MSE 0.556.

Boosting Is Gradient Descent

Loss $L(\mathbf{f}) = \sum_i \frac{1}{2}(Y_i - f_i)^2$ over the vector of fitted values:

$$-\frac{\partial L}{\partial f_i} = Y_i - f_i \quad \text{the residual is the negative gradient}$$

$$\hat{f}^{(m)} = \hat{f}^{(m-1)} + \eta \hat{t}_m(x) \quad \hat{t}_m: \text{ a tree fit to the residuals}$$

- Each tree is small (depth 3 here) and shrunk by η (0.05 here): many weak steps rather than a few big ones (Friedman 2001).
- Any differentiable loss works: logistic for binary Y , Poisson for counts (Week 4); the residual becomes the gradient of that loss.
- The number of steps is the complexity dial; stop when CV error turns up. GSS: about a few hundred rounds.
- **xgboost** adds penalties on leaf values and is the usual default for tables of data (Chen and Guestrin 2016).

One Benchmark: Log Earnings in the GSS

70% training / 30% test, $n = 3,509$; 25 main-effect columns, 294 in the expanded design; tuning by CV inside the training set (figs/w12_gss.R):

test R^2	training $n = 2,456$	training $n = 300$
training mean	0.000	-0.002
OLS, 25 main effects	0.338	0.269
OLS, 294 terms	0.282	-1,368
ridge, 294 terms	0.337	0.288
lasso, 294 terms	0.344	0.305
elastic net, 294 terms	0.345	0.297
pruned tree	0.313	0.194
random forest	0.373	0.332
boosting	0.381	0.322
neural network (1 layer)	0.342	0.256

Large n : ensembles win, modestly. Small n : regularization is everything; the unpenalized 294-term model is far worse than predicting the mean. Most of the variation in earnings stays unpredicted.

Combining Learners: Stacking

No method wins everywhere: lasso won the small-sample GSS race among linear models, the forest won overall. **Stacking** lets the data choose a mix (van der Laan, Polley, and Hubbard 2007):

1. Fit each candidate $\hat{f}_1, \dots, \hat{f}_M$ with K -fold cross-validation; keep the out-of-fold predictions $\hat{f}_m^{(-k)}(X_i)$.
 2. Regress Y_i on the out-of-fold predictions with non-negative weights summing to one: $\min_a \sum_i (Y_i - \sum_m a_m \hat{f}_m^{(-k)}(X_i))^2$.
 3. Predict with $\sum_m \hat{a}_m \hat{f}_m$, each refit on all data.
- Out-of-fold predictions keep a learner that overfits from getting weight it did not earn.
 - Asymptotically at least as good as the best single candidate (the “super learner” oracle result). Next week it supplies the nuisance functions inside DML.

Binary Outcomes: Loss, Thresholds, Calibration

For $Y \in \{0, 1\}$ every method above predicts $\hat{p}(x) = \widehat{\Pr}(Y = 1 \mid x)$: lasso and ridge on the logistic log-likelihood (Week 3), trees on the Gini index or deviance, boosting on the logistic loss.

- **Classification is a second step:** predict 1 when $\hat{p}(x) > c$. The cutoff c is a decision about costs (a missed dropout vs. an unneeded intervention), not a statistical quantity; 0.5 is rarely right.
- **Ranking:** the AUC is the chance a random positive case gets a higher $\hat{p}$ than a random negative one; it ignores the cutoff and the level of $\hat{p}$.
- **Calibration:** among cases with $\hat{p} \approx 0.3$, do 30% have $Y = 1$? Flexible models often rank well and calibrate badly; check a calibration plot on held-out data.
- **Rare outcomes:** accuracy is useless (predict “no” for everyone); report AUC, calibration, and precision among the flagged.

Neural Nets

The Goal: Learn the Features Too

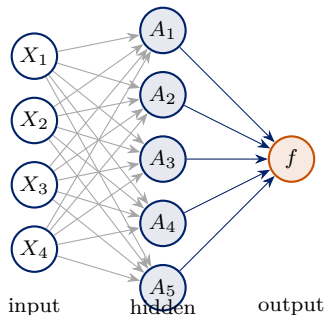
Lasso chooses among features we built (interactions, squares). Trees build features by splitting. A neural network **learns its own features** as smooth functions of the inputs, then regresses on them.

Plan:

1. One hidden layer: the model, activations, and why nonlinearity matters.
2. What a network can represent: an interaction from two neurons; universal approximation; counting parameters.
3. Fitting: gradient descent, backpropagation, stochastic gradients.
4. Regularizing: weight decay, dropout, early stopping.
5. A network on the GSS.

Notation follows ISL Ch. 10: inputs $X_1, \dots, X_p$, K hidden units, activation $\sigma(\cdot)$, weights w , output coefficients β .

A Single Hidden Layer



$$A_k = \sigma\left(w_{k0} + \sum_{j=1}^p w_{kj} X_j\right), \quad k = 1, \dots, K$$
$$f(X) = \beta_0 + \sum_{k=1}^K \beta_k A_k$$

- Each hidden unit is a linear index of the inputs pushed through a fixed nonlinear σ : a logistic regression without an outcome.
- The output is a linear regression on the K learned features.
- w and β are chosen together to minimize $\sum_i (Y_i - f(X_i))^2$.

Activations, and Why They Must Be Nonlinear

$$\text{sigmoid: } \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\text{ReLU: } \sigma(z) = \max(0, z)$$

Suppose $\sigma(z) = z$ instead:

$$\begin{aligned} f(X) &= \beta_0 + \sum_k \beta_k (w_{k0} + \sum_j w_{kj} X_j) && \text{substitute the identity} \\ &= \underbrace{(\beta_0 + \sum_k \beta_k w_{k0})}_{\text{intercept}} + \sum_j \underbrace{(\sum_k \beta_k w_{kj})}_{\text{slope on } X_j} X_j && \text{regroup} \end{aligned}$$

- A linear network of any width or depth is one linear regression. All the flexibility comes from σ .
- ReLU is the modern default: cheap to compute, gradient 1 where active. With the intercept w_{k0} , each unit is a hinge at its own location: ReLU networks are piecewise-linear functions.

Two Neurons Make an Interaction

Two inputs, two hidden units, $\sigma(z) = z^2$; weights $w_1 = (0, 1, 1)$, $w_2 = (0, 1, -1)$, $\beta = (0, \frac{1}{4}, -\frac{1}{4})$:

$$A_1 = (X_1 + X_2)^2, \quad A_2 = (X_1 - X_2)^2 \quad \text{the two hidden units}$$

$$f(X) = \frac{1}{4}(X_1 + X_2)^2 - \frac{1}{4}(X_1 - X_2)^2 \quad \text{output layer}$$

$$= \frac{1}{4}[(X_1^2 + 2X_1X_2 + X_2^2) - (X_1^2 - 2X_1X_2 + X_2^2)] = X_1X_2 \quad \text{expand}$$

- The interaction was never typed in. The network built it from linear indices and a nonlinearity.
- **Universal approximation:** with a non-polynomial activation and enough hidden units, one hidden layer approximates any continuous function on a bounded set (Hornik, Stinchcombe, and White 1989). Existence, not a recipe: it says nothing about how many units, or how to find the weights.

Deeper Networks, and Counting Parameters

Stack layers: layer ℓ treats layer $\ell - 1$'s activations as inputs,

$$A_m^{(\ell)} = \sigma\left(w_{m0}^{(\ell)} + \sum_k w_{mk}^{(\ell)} A_k^{(\ell-1)}\right).$$

network	count	parameters
GSS, 25 inputs, 2 hidden units	$(25 + 1) \cdot 2 + (2 + 1)$	55
MNIST digits: $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$	$785 \cdot 256 + 257 \cdot 128 + 129 \cdot 10$	235,146

- Depth builds features of features (edges, then shapes, then digits), often with far fewer units than one wide layer would need.
- 235,146 parameters for 60,000 training images: more parameters than cases, and still it predicts well. Regularization and the fitting algorithm, not the parameter count, control the effective complexity.

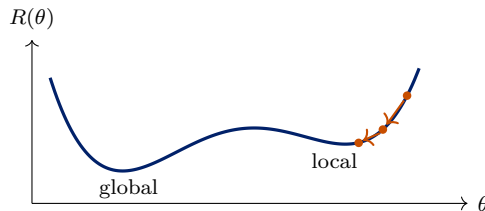
Fitting: Gradient Descent

Collect all weights in θ ; minimize $R(\theta) = \frac{1}{2} \sum_i (Y_i - f_\theta(X_i))^2$.

- Not convex: many local minima and saddle points; no closed form.
- Start from small random θ^0 and step downhill:

$$\theta^{t+1} = \theta^t - \rho \nabla R(\theta^t),$$

with **learning rate** ρ : too large overshoots, too small crawls.



Started on the right, descent settles in the local minimum. Random restarts and averaging (5 starts for the GSS network) are the usual defences.

Backpropagation: the Chain Rule, Organized

One case, one hidden layer; $z_k = w_{k0} + \sum_j w_{kj}X_j$, $A_k = \sigma(z_k)$, residual $r = Y - f$, loss $R = \frac{1}{2}r^2$:

$$\frac{\partial R}{\partial \beta_k} = \frac{\partial R}{\partial f} \cdot \frac{\partial f}{\partial \beta_k} = -r \cdot A_k \quad \text{output weights}$$

$$\frac{\partial R}{\partial w_{kj}} = \frac{\partial R}{\partial f} \cdot \frac{\partial f}{\partial A_k} \cdot \frac{\partial A_k}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_{kj}} = -r \cdot \beta_k \cdot \sigma'(z_k) \cdot X_j \quad \text{hidden weights}$$

- Both gradients start from the residual r : compute f forward, then pass r backward, multiplying by each layer's local derivative.
- Deep networks repeat the second line once per layer. If σ' is small (sigmoid in its flat tails), products of many small numbers vanish: one reason ReLU replaced sigmoid.

Stochastic Gradients, and Regularization

Stochastic gradient descent: use a random mini-batch B_t per step,

$$\theta^{t+1} = \theta^t - \rho \frac{1}{|B_t|} \sum_{i \in B_t} \nabla R_i(\theta^t);$$

one pass through all batches is an *epoch*. Cheap steps, and the noise helps leave poor regions.

Regularization:

- **Weight decay:** add $\lambda \sum \theta_j^2$ (ridge); each step also shrinks θ by $2\rho\lambda\theta$.
- **Dropout:** in each step, zero a random share ϕ of the hidden units and scale the rest by $1/(1 - \phi)$; no unit can rely on another.
- **Early stopping:** watch error on a validation set; stop when it rises. For linear models, stopping gradient descent early acts like a ridge penalty.

Tune architecture (layers, units), regularization (λ , ϕ), and optimization (learning rate, batch size, epochs), usually on a validation set.

A Network on the GSS

One hidden layer (`nnet`), inputs standardized; hidden units $\{2, 5, 10, 20\}$ and weight decay $\{0.01, 0.1, 1\}$ by 5-fold CV; average of 5 random starts (`figs/w12_gss.R`).

- CV picks the **smallest** network: 2 hidden units, decay 0.1, 55 parameters.
- Test R^2 0.342: the same as lasso (0.344), below random forests (0.373) and boosting (0.381). With 300 training cases: 0.256, below OLS.
- Nothing in 25 survey variables needs learned representations; the signal is mostly additive and smooth, and trees capture the few thresholds.

This matches the benchmark literature: on tables of data, tree ensembles still beat deep networks (Grinsztajn, Oyallon, and Varoquaux 2022). Networks win when the input is unstructured: text, images, audio.

Beyond

Representations: Words as Vectors

Goal: turn unstructured inputs into numbers a model can use.

- **Word embeddings:** train a network to predict a word from its neighbours; the hidden-layer weights give each word a vector (say 300 numbers). Words used in similar contexts get similar vectors.
- Directions mean something: $\text{vector}(\textit{rich}) - \text{vector}(\textit{poor})$, averaged over many antonym pairs, defines an affluence dimension; projecting other words onto it measures their class connotation.
- Kozlowski, Taddy, and Evans (2019, ASR): dimensions of class (affluence, education, cultivation) in a century of books, and how their meanings moved.
- **Images:** convolutional networks share weights across locations, so a detector for a shape works anywhere in the image. Gebru et al. (2017): cars in Google Street View images predict neighbourhood income, race, and voting.

Attention and Transformers

A sentence is a sequence of word vectors $x_1, \dots, x_T$. Each position builds a query q_t , key k_t , and value v_t (three learned linear maps of x_t):

$$\text{output}_t = \sum_{s=1}^T \alpha_{ts} v_s, \quad \alpha_{ts} = \frac{\exp(q'_t k_s / \sqrt{d})}{\sum_{s'} \exp(q'_t k_{s'} / \sqrt{d})}$$

- Each word's new representation is a weighted average of all words, with weights from how well its query matches their keys: context decides meaning (*bank* near *river* vs. near *loan*).
- Stack many such layers, train on billions of words to predict the next word, and you have a large language model (Vaswani et al. 2017).
- The softmax weights are the same logic as the multinomial logit (Week 4).

Predictions as Measurements

Sociologists increasingly use model outputs as *variables*: an LLM codes the topic of 100,000 speeches; a classifier labels images. Errors in those labels bias every downstream estimate.

Prediction-powered inference (Angelopoulos et al. 2023): hand-code a random subsample of n cases; the model labels all N . For a mean,

$$\hat{\theta}_{PP} = \underbrace{\frac{1}{N} \sum_{i=1}^N \hat{Y}_i}_{\text{model, everyone}} - \underbrace{\frac{1}{n} \sum_{i=1}^n (\hat{Y}_i - Y_i)}_{\text{model's bias, estimated on the coded cases}}$$

- Unbiased whatever the model's quality; the better the model, the smaller the variance relative to using the n coded cases alone.
- The same correction extends to regressions: Egami, Hinck, Stewart, and Wei (2023). The coded subsample must be random.

Interpretation

Looking Inside a Black Box

- **Permutation importance:** shuffle one variable in the test data and record how much error rises. Correlated variables share credit unpredictably, and impurity-based importance favours variables with many cut points (Strobl et al. 2007).
- **Partial dependence:** average prediction as one variable is set to each value for everyone, $\bar{f}(x_j) = \frac{1}{n} \sum_i \hat{f}(x_j, X_{i,-j})$. It evaluates the model at combinations (a 22-year-old with 30 years of experience) that never occur.
- **SHAP values:** split each prediction into additive contributions using Shapley values from game theory (Lundberg and Lee 2017). Same caveat: the attribution describes the model, not the world.

None of these is a causal effect

They answer “what does this fitted function do when I move this input?” Turning that into “what happens when a person’s input changes” needs Weeks 5–11.

Prediction Intervals Without a Model: Split Conformal

Lei, G'Sell, Rinaldo, Tibshirani, and Wasserman (2018): any $\hat{f}$, any distribution, exchangeable data.

1. Split the sample. Fit $\hat{f}$ on part 1.
2. On part 2 (n_2 cases), compute absolute residuals $R_i = |Y_i - \hat{f}(X_i)|$.
3. Let $\hat{q}$ be the $\lceil (n_2 + 1)(1 - \alpha) \rceil$ -th smallest R_i .
4. Interval for a new case: $\hat{f}(x) \pm \hat{q}$.

Why it works: the new residual is exchangeable with the n_2 calibration residuals, so its rank among them is uniform; the chance it exceeds $\hat{q}$ is at most α . Then

$$\Pr(Y_{\text{new}} \in \hat{f}(X_{\text{new}}) \pm \hat{q}) \geq 1 - \alpha$$

in finite samples, whether or not $\hat{f}$ is any good. A bad $\hat{f}$ gives a wide interval, not a wrong one. Coverage is on average over x , not at each x .

What Changed With Very Flexible Models

- **Double descent** (Belkin, Hsu, Ma, and Mandal 2019): past the point where a model can fit the training data exactly, test error can fall again as complexity keeps rising. The U-shaped curve is the left half of a bigger picture.
- **Why:** among the many exact fits, the fitting algorithm picks the smallest-norm one, an implicit ridge penalty; with very many parameters, that interpolant can predict well (Hastie, Montanari, Rosset, and Tibshirani 2022).
- **For tables of social data** (rows of people, columns of variables), tree ensembles still beat deep networks on most benchmarks (Grinsztajn, Oyallon, and Varoquaux 2022). Deep learning earns its keep on text, images, and audio.
- The bias–variance logic still governs choices; “complexity” is just not the number of parameters.

Social Science

In the Literature: the Fragile Families Challenge

The question. How predictable are life outcomes, given rich data from birth to age 9? **The design.** A mass collaboration: 4,242 families, 12,942 variables; outcomes at age 15 released for half the families; 160 teams submitted predictions, scored on a held-out set never seen by anyone.

outcome	best R^2_{holdout}	outcome	best R^2_{holdout}
material hardship	0.23	eviction	0.05
child's GPA	0.19	caregiver job training	0.05
child's grit	0.06	caregiver layoff	0.03

- The best of 160 teams was only slightly better than a four-variable linear benchmark chosen by a domain expert (mother's race, marital status, and education at birth; the outcome measured at age 9).
- Errors depended on the *family*, not on the method: the same families were mispredicted by everyone.

The Fragile Families Challenge: What It Means

- **A ceiling, not a failure:** with these data, most variation in life outcomes is unpredictable from childhood measures. That is a substantive finding about stratification, measured honestly.
- **The holdout did the work:** 160 teams tuning against a public leaderboard would have overfit it; a set opened once, at the end, could not be.
- **GSS echo:** boosting reached $R^2 = 0.38$ for log earnings, OLS 0.34. Flexible methods add little when the signal is mostly additive and the noise is large.
- **Prediction as a benchmark for theory:** fit a flexible model on the same covariates as your theoretical model; a large gap in out-of-sample fit says the functional form is wrong (Verhagen 2024).

In the Literature: Machine Predictions and Bail (Kleinberg et al. 2018)

The question. Judges decide pretrial release by predicting crime if released. Could an algorithm predict better?

The data. New York City arrests 2008–13; 554,689 cases in the working sample. Judges release 73.6%; 11.3% of the released commit a crime before trial.

The results (held-out test set):

- Same release rate as the judges: crime falls 24.7%.
- Same crime rate: detention falls from 26.4% to 15.3% (a 41.9% cut).
- Both can be achieved while reducing racial disparities in detention.

The catch: selective labels. Crime is observed only for defendants the judge released. The algorithm's prediction for the detained cannot be checked directly.

Kleinberg et al.: Handling Selective Labels

- Cases are assigned to judges as good as randomly within courthouse-day cells (the judge design of Week 7).
- **Contraction:** start from the defendants released by the most lenient judges, whose outcomes are mostly observed. Detain more of them in order of predicted risk, and compare with what stricter judges achieved by detaining more.
- Stricter (second-quintile) judges cut crime 9.9% relative to the most lenient; the algorithm matches that cut while jailing far fewer people.
- The headline imputations still assume selection on observables for the released; contraction is the check that does not.

A prediction problem with a causal core: the policy question is what happens if the rule changes, and that needs a design, not only a better $\hat{f}$.

A Biased Algorithm, and Why

A commercial algorithm ranks patients for a care-management program (6,079 Black and 43,539 White patients). Above the 97th percentile of the score, patients are enrolled automatically.

- At the same score, Black patients are sicker: at the 97th percentile they have 26.3% more chronic conditions (4.8 vs. 3.8).
- **Cause: the label.** The algorithm predicts *cost*. At the same health, Black patients generate \$1,801 less cost a year, through unequal access to care. The prediction is accurate; the target is wrong.
- Equalizing health at the threshold would raise the Black share of auto-enrolled patients from 17.7% to 46.5%.
- Retraining on a label combining health and cost cut the excess chronic conditions by 84%.

The sociological lesson: what is measured, and for whom, is built into the outcome variable before any model is fit.

Fairness Criteria Conflict

A risk score S , groups a , outcome Y . Three reasonable requirements:

1. **Calibration** within groups: $\Pr(Y = 1 \mid S = s, a) = s$.
2. **Balance for the negative class**: equal mean S among those with $Y = 0$.
3. **Balance for the positive class**: equal mean S among those with $Y = 1$.

Kleinberg, Mullainathan, and Raghavan (2017): all three hold only with perfect prediction or equal base rates. For a yes/no rule (Chouldechova 2017),

$$\text{FPR} = \frac{p}{1-p} \cdot \frac{1 - \text{PPV}}{\text{PPV}} \cdot (1 - \text{FNR}),$$

with p the base rate: equal PPV across groups with different p forces different error rates. COMPAS: recidivism 51% among Black and 39% among White defendants, so the two sides of that debate were each right about a different criterion.

Practice

What ML Buys, and What It Does Not ★

Buys

- Prediction with many predictors, including $p > n$.
- Automatic nonlinearity and interactions, no functional form to guess.
- Honest out-of-sample performance estimates.

Does not buy

- **Causal interpretation.** A variable importance score is not an effect.
- Valid standard errors for the coefficients it selects.
- Extrapolation: trees do return a prediction for out-of-range inputs, but only the boundary leaf's constant, so they cannot extend a trend.
- Freedom from the identification problems of Weeks 5–11.

What ML Buys, and What It Does Not ★ (cont.)

The genuine sociological uses

Prediction as the goal (screening, triage, forecasting); *measurement* (constructing variables from text or images); *describing heterogeneity*; and **estimating nuisance functions inside a causal estimator**, which is next week.

Looking Ahead

- Every method today trades bias for variance, tuned by cross-validation.
- That trade is exactly why we cannot drop these estimates into a regression and read off a coefficient: they are *deliberately biased* and converge slowly.
- **Next week:** what has to change before an estimate this biased can be used as an ingredient in a causal estimator, and what it buys us when it can.

Problem Set 5

Assigned today, due Monday November 23. Covers Weeks 10–11.

Reading

ISL Chapters 5, 6, and 8; Molina and Garip (2019). Optional: ISL Chapter 10 (deep learning); Kleinberg et al. (2015) on prediction policy problems.

References

- Angelopoulos, Anastasios N., Stephen Bates, Clara Fannjiang, Michael I. Jordan, and Tijana Zrnica. 2023. “Prediction-Powered Inference.” *Science* 382(6671): 669–674.
- Athey, Susan and Guido W. Imbens. 2019. “Machine Learning Methods That Economists Should Know About.” *Annual Review of Economics* 11: 685–725.
- Bates, Stephen, Trevor Hastie, and Robert Tibshirani. 2024. “Cross-Validation: What Does It Estimate and How Well Does It Do It?” *Journal of the American Statistical Association* 119(546): 1434–1445.
- Belkin, Mikhail, Daniel Hsu, Siyuan Ma, and Soumik Mandal. 2019. “Reconciling Modern Machine-Learning Practice and the Classical Bias–Variance Trade-Off.” *PNAS* 116(32): 15849–15854.
- Breiman, Leo. 2001. “Random Forests.” *Machine Learning* 45(1): 5–32.
- Chen, Tianqi and Carlos Guestrin. 2016. “XGBoost: A Scalable Tree Boosting System.” *Proceedings of KDD '16*: 785–794.

References (cont.)

- Chouldechova, Alexandra. 2017. “Fair Prediction with Disparate Impact.” *Big Data* 5(2): 153–163.
- Egami, Naoki, Musashi Hinck, Brandon M. Stewart, and Hanying Wei. 2023. “Using Imperfect Surrogates for Downstream Inference.” *Advances in Neural Information Processing Systems* 36.
- Friedman, Jerome H. 2001. “Greedy Function Approximation: A Gradient Boosting Machine.” *Annals of Statistics* 29(5): 1189–1232.
- Gebru, Timnit, et al. 2017. “Using Deep Learning and Google Street View to Estimate the Demographic Makeup of Neighborhoods across the United States.” *PNAS* 114(50): 13108–13113.
- Grinsztajn, Léo, Edouard Oyallon, and Gaël Varoquaux. 2022. “Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?” *NeurIPS Datasets and Benchmarks*.
- Hastie, Trevor, Andrea Montanari, Saharon Rosset, and Ryan J. Tibshirani. 2022. “Surprises in High-Dimensional Ridgeless Least Squares Interpolation.” *Annals of Statistics* 50(2): 949–986.
- Hastie, Trevor, Robert Tibshirani, and Jerome Friedman. 2009. *The Elements of Statistical Learning*, 2nd ed. Springer.

References (cont.)

- Hoerl, Arthur E. and Robert W. Kennard. 1970. “Ridge Regression: Biased Estimation for Nonorthogonal Problems.” *Technometrics* 12(1): 55–67.
- Hornik, Kurt, Maxwell Stinchcombe, and Halbert White. 1989. “Multilayer Feedforward Networks Are Universal Approximators.” *Neural Networks* 2(5): 359–366.
- James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. 2021. *An Introduction to Statistical Learning*, 2nd ed. Springer. [Ch. 5, 6, 8, 10]
- Kleinberg, Jon, Himabindu Lakkaraju, Jure Leskovec, Jens Ludwig, and Sendhil Mullainathan. 2018. “Human Decisions and Machine Predictions.” *Quarterly Journal of Economics* 133(1): 237–293.
- Kleinberg, Jon, Jens Ludwig, Sendhil Mullainathan, and Ziad Obermeyer. 2015. “Prediction Policy Problems.” *American Economic Review: Papers & Proceedings* 105(5): 491–495.
- Kleinberg, Jon, Sendhil Mullainathan, and Manish Raghavan. 2017. “Inherent Trade-Offs in the Fair Determination of Risk Scores.” *ITCS 2017*, LIPIcs 67: 43:1–43:23.

References (cont.)

- Kozlowski, Austin C., Matt Taddy, and James A. Evans. 2019. “The Geometry of Culture: Analyzing the Meanings of Class through Word Embeddings.” *American Sociological Review* 84(5): 905–949.
- Lei, Jing, Max G’Sell, Alessandro Rinaldo, Ryan J. Tibshirani, and Larry Wasserman. 2018. “Distribution-Free Predictive Inference for Regression.” *Journal of the American Statistical Association* 113(523): 1094–1111.
- Lundberg, Scott M. and Su-In Lee. 2017. “A Unified Approach to Interpreting Model Predictions.” *Advances in Neural Information Processing Systems* 30.
- Molina, Mario and Filiz Garip. 2019. “Machine Learning for Sociology.” *Annual Review of Sociology* 45: 27–45.
- Mullainathan, Sendhil and Jann Spiess. 2017. “Machine Learning: An Applied Econometric Approach.” *Journal of Economic Perspectives* 31(2): 87–106.
- Obermeyer, Ziad, Brian Powers, Christine Vogeli, and Sendhil Mullainathan. 2019. “Dissecting Racial Bias in an Algorithm Used to Manage the Health of Populations.” *Science* 366(6464): 447–453.

References (cont.)

- Salganik, Matthew J., Ian Lundberg, Alexander T. Kindel, et al. 2020. “Measuring the Predictability of Life Outcomes with a Scientific Mass Collaboration.” *PNAS* 117(15): 8398–8403.
- Strobl, Carolin, Anne-Laure Boulesteix, Achim Zeileis, and Torsten Hothorn. 2007. “Bias in Random Forest Variable Importance Measures.” *BMC Bioinformatics* 8: 25.
- Tibshirani, Robert. 1996. “Regression Shrinkage and Selection via the Lasso.” *Journal of the Royal Statistical Society Series B* 58(1): 267–288.
- van der Laan, Mark J., Eric C. Polley, and Alan E. Hubbard. 2007. “Super Learner.” *Statistical Applications in Genetics and Molecular Biology* 6(1): 25.
- Vaswani, Ashish, et al. 2017. “Attention Is All You Need.” *Advances in Neural Information Processing Systems* 30.
- Verhagen, Mark D. 2024. “Incorporating Machine Learning into Sociological Model-Building.” *Sociological Methodology* 54(2): 217–268.

References (cont.)

- Zou, Hui and Trevor Hastie. 2005. “Regularization and Variable Selection via the Elastic Net.” *Journal of the Royal Statistical Society Series B* 67(2): 301–320.
- Zou, Hui, Trevor Hastie, and Robert Tibshirani. 2007. “On the ‘Degrees of Freedom’ of the Lasso.” *Annals of Statistics* 35(5): 2173–2192.